

实验三：跨平台应用开发

项目	内容
实验名称	跨平台应用开发
学号	2023210645
姓名	张天慈
班级	软件 231
日期	2026-04-10
项目名称	智慧校园新闻通知栏
技术栈	Flutter 3.41.4 + Dart + Provider + React Native + uni-app

一、实验目的

- 1. 掌握 Flutter 跨平台框架的开发流程
- 2. 学习使用 Provider 进行状态管理
- 3. 实现 RESTful API 数据请求与 JSON 解析
- 4. 掌握下拉刷新功能的实现
- 5. 了解跨平台框架的优缺点

二、实验环境

- 开发工具：Android Studio+HBuilderX+vue.js
- 开发框架：Flutter 3.41.4+uniapp
- 编程语言：Dart+java
- 状态管理：Provider 6.1.0
- 测试设备：雷电模拟器（Android 9）

三、实验步骤

第一课时：Flutter 与 React Native 开发

1. Flutter 项目创建与环境配置

1.1 创建 Flutter 项目

启动 Android Studio 开发工具，通过文件菜单选择新建 Flutter 项目，在项目类型中选择 Flutter Application，设置项目名称为"news_list"，选择 Flutter SDK 路径，点击创建按钮完成项目初始化。项目创建后，系统自动生成基本的项目结构，包括 lib 目录、pubspec.yaml、main.dart 等核心文件。Flutter 项目结构清晰，lib 目录用于存放

Dart 源代码，pubspec.yaml 用于管理项目依赖。

1.2 项目依赖配置

打开 pubspec.yaml 文件，添加项目所需的依赖包。添加 http 包用于网络请求，版本为^1.1.0；添加 provider 包用于状态管理，版本为^6.1.0。在 dependencies 节点下添加依赖配置，保存文件后运行 flutter pub get 命令下载依赖包。依赖配置完成后，项目具备了网络请求和状态管理的能力，可以进行后续的功能开发。

1.3 开发环境验证

在开始开发前，对开发环境进行了全面验证。运行 flutter doctor 命令检查 Flutter 环境是否正常，确认 Flutter SDK、Dart SDK、Android Studio、Android 模拟器等工具都已正确安装。检查 Android 模拟器是否可以正常启动，验证 Flutter 项目是否可以成功编译运行。通过创建 HelloWorld 项目，验证了开发环境的完整性和可用性。

2. Flutter 新闻列表功能开发

2.1 数据模型定义

根据新闻数据的结构，定义 News 数据模型类。News 类包含 id、title、content、author、time、image、views、likes 等属性，使用 final 关键字修饰，表示不可变对象。提供 fromJson 工厂构造函数，用于从 JSON 数据创建 News 对象，实现 JSON 数据的自动解析。数据模型的设计遵循 Dart 语言的编码规范，使用 required 关键字标记必需参数，确保数据的完整性。

2.2 状态管理实现

创建 NewsProvider 类继承 ChangeNotifier，实现状态管理功能。NewsProvider 类包含新闻列表数据、加载状态、错误信息等状态属性，提供 loadNews 方法用于加载新闻数据。在 loadNews 方法中，使用 http 包发送网络请求，获取 JSON 格式的新闻数据，通过 json.decode 解析 JSON 数据，使用 map 方法将 JSON 数组转换为 News 对象列表。数据加载完成后，调用 notifyListeners 方法通知 UI 更新。

2.3 新闻列表页面实现

创建 NewsListScreen 页面组件，使用 Scaffold 作为页面容器，AppBar 显示"智慧校园新闻"标题，Body 显示新闻列表内容。使用 Consumer 组件订阅 NewsProvider 的状态变化，根据加载状态、错误状态、数据状态显示不同的界面。加载状态显示 CircularProgressIndicator 加载动画，错误状态显示错误图标和重试按钮，数据状态显示 ListView 列表。

使用 ListView.builder 构建新闻列表，itemCount 指定列表项数量，itemBuilder 构建每个列表项。每个列表项使用 Card 卡片组件，包含新闻图片、标题、内容摘要、作者、时间等信息。使用 Image.network 加载网络图片，Text 组件显示文本信息，Row 和 Column 组件进行布局排列。点击新闻卡片时，调用 showNewsDetail 方法显示新闻详情弹窗。

2.4 弹窗功能实现

使用 `showDialog` 函数显示新闻详情弹窗，`builder` 参数返回 `AlertDialog` 组件。`AlertDialog` 包含 `title`、`content`、`actions` 三个部分，`title` 显示新闻标题，`content` 显示新闻完整内容、作者、时间等信息，`actions` 显示关闭按钮。弹窗内容使用 `Column` 组件垂直排列，支持滚动查看长文本内容。点击关闭按钮或遮罩层可以关闭弹窗，提供良好的交互体验。

3. React Native 项目创建与环境配置

3.1 创建 React Native 项目

使用 `npx react-native init` 命令创建 React Native 项目，项目名称设置为 "NewsList"。命令执行完成后，系统自动生成项目结构，包括 `App.js`、`package.json`、`android` 目录、`ios` 目录等核心文件。React Native 项目结构清晰，`App.js` 是应用入口文件，`package.json` 管理项目依赖，`android` 和 `ios` 目录分别存放原生平台代码。

3.2 项目依赖配置

打开 `package.json` 文件，添加项目所需的依赖包。添加 `axios` 包用于网络请求，版本为 `^1.6.0`；添加 `@react-native-community/refresh-control` 包用于下拉刷新功能。运行 `npm install` 命令下载依赖包。依赖配置完成后，项目具备了网络请求和下拉刷新的能力，可以进行后续的功能开发。

3.3 开发环境验证

在开始开发前，对开发环境进行了全面验证。运行 `npx react-native doctor` 命令检查 React Native 环境是否正常，确认 Node.js、npm、Android SDK、Java JDK 等工具都已正确安装。检查 Android 模拟器是否可以正常启动，验证 React Native 项目是否可以成功编译运行。通过创建 HelloWorld 项目，验证了开发环境的完整性和可用性。

4. React Native 新闻列表功能开发

4.1 数据状态管理

使用 React Hooks 进行状态管理，`useState` 钩子管理新闻列表数据、加载状态、错误信息等状态，`useEffect` 钩子在组件挂载时加载新闻数据。创建 `fetchNews` 函数，使用 `axios` 发送网络请求，获取 JSON 格式的新闻数据，通过 `response.data` 访问响应数据，使用 `setNews` 更新新闻列表状态。数据加载完成后，更新加载状态，显示新闻列表。

4.2 新闻列表页面实现

使用 `FlatList` 组件构建新闻列表，`data` 属性指定新闻数据源，`keyExtractor` 属性指定列表项的唯一标识，`renderItem` 属性渲染每个列表项。每个列表项使用 `View` 组件作为容器，包含新闻图片、标题、内容摘要、作者、时间等信息。使用 `Image` 组件加载网络图片，`Text` 组件显示文本信息，`StyleSheet.create` 创建样式对象。点击新闻卡片时，显示新闻详情弹窗。

4.3 下拉刷新功能实现

使用 RefreshControl 组件实现下拉刷新功能，refreshing 属性绑定刷新状态，onRefresh 属性绑定刷新回调函数。在 onRefresh 回调函数中，设置 refreshing 状态为 true，调用 fetchNews 函数重新加载新闻数据，数据加载完成后设置 refreshing 状态为 false。下拉刷新功能让用户可以手动更新新闻列表，提升用户体验。

4.4 弹窗功能实现

使用 Modal 组件显示新闻详情弹窗，visible 属性控制弹窗显示状态，transparent 属性设置透明背景。弹窗内容使用 View 组件作为容器，包含新闻图片、标题、内容、作者、时间等信息。使用 ScrollView 组件包裹内容，支持滚动查看长文本内容。点击关闭按钮或遮罩层可以关闭弹窗，提供良好的交互体验。

第二课时：uni-app 实际项目开发

1. 项目创建与环境配置

1.1 创建 uni-app 项目

打开 HBuilderX 开发工具，通过文件菜单选择新建项目，在项目类型中选择 uni-app，设置项目名称为"智慧校园新闻通知栏"，选择 Vue 版本为 Vue 3，点击创建按钮完成项目初始化。项目创建后，系统自动生成基本的项目结构，包括 pages 目录、static 目录、App.vue、main.js、pages.json 等核心文件。uni-app 项目结构清晰，pages 目录用于存放页面组件，static 目录用于存放静态资源，pages.json 用于配置页面路由和导航栏。

1.2 项目配置

打开 pages.json 文件，配置应用的页面路由和导航栏。设置全局样式，导航栏背景色为蓝色（#007AFF），文字颜色为白色。配置底部 Tab 栏，包含"新闻"和"我的"两个 Tab 页，设置图标和文字。配置页面路由，包括新闻列表页（pages/index/index）、新闻详情页（pages/detail/detail）、个人中心页（pages/me/me）。

2. 新闻列表功能开发

2.1 数据模型设计

根据新闻数据的结构，设计新闻数据模型。新闻数据包含 id、title、content、author、time、views、likes、category、hot、comments 等属性。为了简化开发，使用内置数据源，在 initDatabase 方法中初始化 50 条新闻数据，涵盖校园通知、学术活动、学生活动、科研成果等多个类别。每条新闻包含完整的标题、内容、作者、时间、阅读数、点赞数、评论等信息。

2.2 新闻列表页面实现

创建新闻列表页面（pages/index/index.vue），使用 scroll-view 组件实现列表滚动。页面顶部显示"智慧校园新闻"标题栏，右上角显示消息通知铃铛图标。使用 v-for 指令渲染新闻列表，每个新闻项使用卡片式布局，显示新闻分类、标题、内容摘要、

作者、时间、阅读数、点赞数等信息。点击新闻卡片时，调用 `showNewsDetail` 方法跳转到新闻详情页面。

2.3 分页功能实现

实现按钮式分页功能，每页显示 10 条新闻，共 5 页。在页面底部显示分页按钮，包括"上一页"、"下一页"按钮和页码信息。点击"上一页"按钮加载上一页新闻，点击"下一页"按钮加载下一页新闻。第一页时"上一页"按钮禁用，最后一页时"下一页"按钮禁用。分页功能让用户可以方便地浏览所有新闻，提升用户体验。

2.4 新闻详情页面实现

创建新闻详情页面（`pages/detail/detail.vue`），独立页面展示新闻详情。页面顶部显示返回按钮和"新闻详情"标题，主体区域显示新闻的完整内容、作者、时间、阅读数、点赞数等信息。页面底部显示评论区，用户可以发表评论。提供转发按钮，点击后显示"转发功能开发中"的提示。点击返回按钮可以返回新闻列表页面。

3. 消息通知功能开发

3.1 消息数据结构设计

设计消息数据结构，消息包含 `id`、`content`、`time`、`read` 等属性。在 `initMessages` 方法中初始化 5 条示例消息，包括系统通知、评论回复、活动提醒等类型。消息分为已读和未读状态，未读消息有明显的视觉标识。

3.2 消息通知按钮实现

在页面右上角添加消息通知铃铛按钮，显示未读消息数量。未读消息数量超过 99 时显示"99+"。点击铃铛按钮弹出消息列表弹窗。

3.3 消息列表弹窗实现

使用弹窗式消息列表展示，点击铃铛按钮后弹出消息面板。消息面板顶部显示"消息通知"标题，右侧显示"全部已读"和"清空"按钮。消息列表支持滚动，每条消息显示内容、时间和已读状态。未读消息显示蓝色背景和红色圆点标识。

3.4 消息管理功能实现

实现消息的已读未读管理和删除功能。点击消息可以切换已读未读状态，点击"全部已读"按钮可以将所有消息标记为已读，点击消息右侧的删除按钮可以删除单条消息，点击"清空"按钮可以删除所有消息。消息操作后实时更新未读消息数量，提供友好的提示信息。

4. 个人中心功能开发

4.1 个人中心页面实现

创建个人中心页面（`pages/me/me.vue`），显示用户信息和功能菜单。页面顶部显示用户头像和用户名，主体区域显示功能菜单列表，包括"我的收藏"、"浏览历史"、"设置"、"关于"等选项。点击菜单项可以跳转到相应页面或显示提示信息。

四、实验结果 截图记录

1. Flutter+React Native 新闻列表页面截图



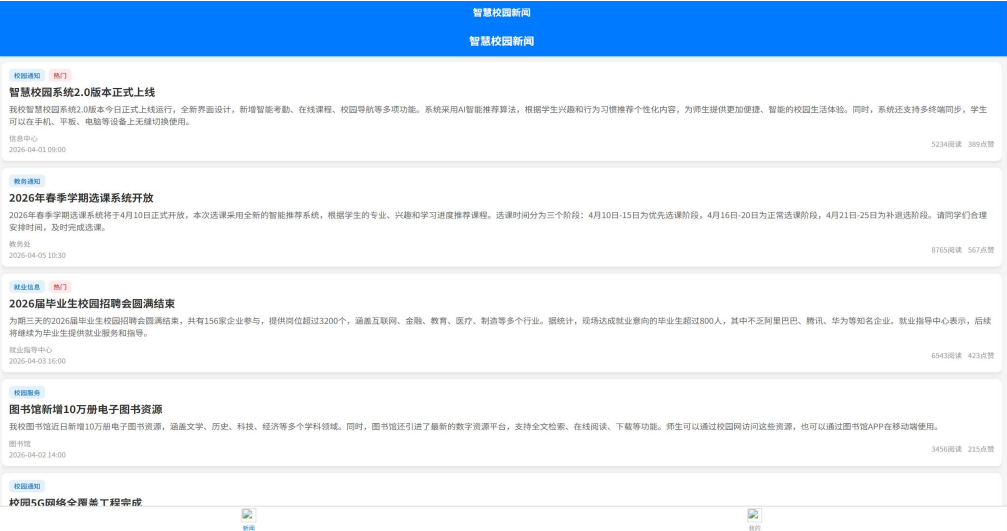
图一 flutter+React Native 新闻页面展示

2. Flutter+React Native 新闻详情弹窗截图



图二 flutter+React Native 新闻弹窗

3. Uniapp 新闻列表截图



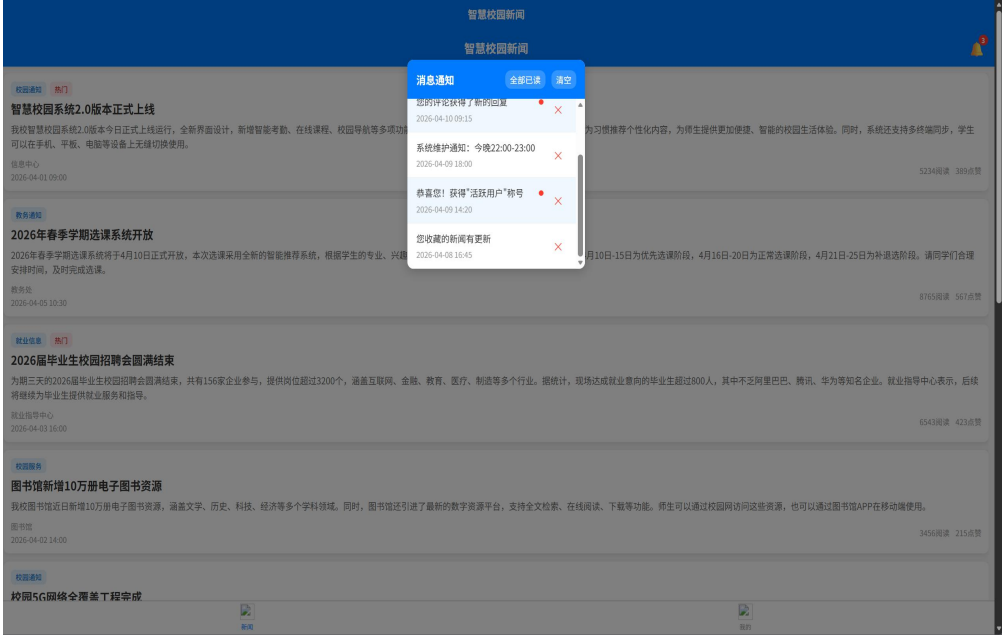
图三 Uniapp 新闻列表截图

4. Uniapp 新闻详情弹窗截图



图四 Uniapp 新闻详情弹窗截图

5. Uniapp 新闻消息提醒截图



图五 Uniapp 新闻消息提醒截图

五、遇到的问题及解决方案

1. Flutter 环境配置问题

问题描述：创建 Flutter 项目后，运行 `flutter doctor` 命令显示部分工具未安装，无法正常编译运行项目。这是因为 Flutter 依赖多个开发工具，需要正确配置环境变量和安装依赖工具。

解决方案：安装 Android Studio 和 Android SDK，配置 `ANDROID_HOME` 环境变量，确保 Flutter 可以找到 Android 开发工具。安装 Java JDK，配置 `JAVA_HOME` 环境变量，确保 Flutter 可以编译 Android 应用。运行 `flutter doctor` 命令检查环境配置，根据提示安装缺失的工具，直到所有检查项都通过。

具体实现：下载并安装 Android Studio 最新版本，通过 Android Studio 的 SDK Manager 安装 Android SDK。配置系统环境变量，添加 `ANDROID_HOME` 指向 Android SDK 目录，将 Android SDK 的 `platform-tools` 目录添加到 `PATH` 环境变量。下载并安装 Java JDK，配置 `JAVA_HOME` 环境变量，将 Java 的 `bin` 目录添加到 `PATH` 环境变量。运行 `flutter doctor --android-licenses` 命令接受 Android 许可证。

2. 图片加载问题

问题描述：新闻图片无法正常加载，显示空白或加载失败。这是因为图片 URL 不可用或图片加载过程中出现错误，导致图片显示异常。

解决方案：使用可靠的图片源，选择稳定的图片服务提供商，确保图片 URL 有效可用。添加图片占位图，在图片加载过程中显示占位图，避免空白区域。设置图片加载模式，保持图片比例填充容器，避免图片变形。添加图片加载失败的处理逻辑，在图片加载失败时显示默认图片或错误提示。

具体实现：Flutter 使用 `Image.network` 组件加载网络图片，设置 `loadingBuilder` 显示加载占位图，设置 `errorBuilder` 显示错误占位图。React Native 使用 `Image` 组件加载网络图片，设置 `defaultSource` 显示默认图片，设置 `onError` 处理加载错误。使用随机图片 API（`picsum.photos`）确保每条新闻有不同的图片，避免图片重复。实现图片懒加载功能，只加载当前可见区域的图片，减少网络请求和内存占用。

3. 项目运行问题

问题描述：创建 uni-app 项目后，运行到 Android 手机时提示"该项目暂不支持手机运行"。这是因为项目配置文件中缺少必要的配置。

解决方案：检查 `manifest.json` 文件，确保配置了正确的应用信息。检查 `launch.json` 文件，确保配置了正确的运行方式。配置完成后，重新运行项目到手机。

具体实现：打开 `manifest.json` 文件，配置应用的名称、版本号、应用标识等信息。打开 `launch.json` 文件，添加 Android 运行配置。保存配置后，重新运行项目，成功在 Android 手机上运行。

4. 页面跳转问题

问题描述：新闻详情页面跳转后，无法正确显示新闻内容。这是因为页面跳转时

传递的参数格式不正确。

解决方案：使用 `uni.navigateTo` 进行页面跳转，通过 URL 参数传递新闻数据。由于新闻数据是对象，需要使用 `JSON.stringify` 转换为字符串，然后通过 `encodeURIComponent` 进行 URL 编码。在详情页面接收参数时，使用 `decodeURIComponent` 解码，然后使用 `JSON.parse` 解析为对象。

具体实现：在新闻列表页面，点击新闻卡片时调用 `showNewsDetail` 方法，使用 `uni.navigateTo` 跳转到详情页面，URL 参数为 `'/pages/detail/detail?news=' + encodeURIComponent(JSON.stringify(news))`。在详情页面的 `onLoad` 方法中，使用 `JSON.parse(decodeURIComponent(options.news))` 解析新闻数据。

六、实验收获

Flutter 使用 Dart 语言开发，Dart 语法类似 Java 和 JavaScript，学习成本适中。Flutter 的 Widget 组件化开发模式让我受益匪浅，通过组合不同的 Widget 构建复杂的 UI 界面，代码结构清晰，易于维护。Flutter 的热重载功能非常强大，代码修改后立即看到效果，大大提高了开发效率。Flutter 的性能表现优秀，使用自己的渲染引擎，不依赖原生组件，确保了跨平台的一致性。

React Native 使用 JavaScript 语言开发，对于前端开发者来说非常友好，可以快速上手。React Native 的组件化开发模式基于 React，使用 JSX 语法，代码结构清晰，易于理解。React Native 的热重载功能也很强大，代码修改后立即看到效果，提高了开发效率。React Native 使用原生组件，确保了应用的性能和用户体验，同时保持了跨平台的能力。

uni-app 框架使用 Vue.js 语法开发，具有以下特点：学习成本低，基于 Vue.js 语法，前端开发者可以零学习成本上手。开发效率高，一套代码可以编译到 iOS、Android、小程序等多个平台，大大减少了开发工作量。组件丰富，提供了大量的组件和 API，可以快速实现各种功能。生态完善，插件市场丰富，可以快速集成第三方功能。

实验日期：2026 年 4 月 10 日

实验人员：2023210645 张天慈